

FUNCTIONS

CSE 231: Printing using the Runtime

Ranjit Jhala April 23, 2026

1 calling "print"

2 "user def"

Printing

Lets add support for **printing** expressions

A stepping stone to *user-defined functions* and *calls*

```
<expr> := ... | (print <expr>)
```

QUIZ: Semantics

Program

(add1 (print 42))

```
(let (x1 100)
  (let (x2 200)
    (block
      (print x1)
      (print x2))))
```

Expected Output

42
43 ← by runtime

100
200
200 ←

Abstract Syntax

```
enum Expr { // ...
  Print(Box<Expr>),
}
```

Evaluation

```
fn eval(expr: &Expr, env: &mut Env) -> Value {
  match expr { // ...
    Expr::Print(e) => {
      }
  }
}
```

see code

Strategy

1. Expose **snek_print** in the **runtime** (start.rs),
2. Compile print to **call** snek_print.

Similar to how we exposed snek_error for tag errors

Exposing `snek_print`

In the Runtime

```
#[export_name = "\x01snek_print"]
fn snek_print(val: i64) -> i64 { ... }
```

In the Assembly

```
global our_code_starts_here
extern snek_print
extern snek_error
...
```

QUIZ: Assembly for `print`

Program

```
(print 42)
```

Assembly

```
mov rax, <42>
mov rdi, rax
call snek_print
```

```
(print e)
```

```
<e>
mov rdi, rax
call snek_print
```

QUIZ: Passing parameters with `rdi`

Like with `snek_error`

- We used `rdi` to pass the param to `snek_print`

Unlike with `snek_error`

- We're coming back!

But *unlike* with `snek_error`, we're coming back!

Can you write a test that will *break* our compiler?

```
; INPUT = 200
(block
  (print 100)
  input)
```

Saving RDI

What were we using `rdi` for before? How can we save it?

<see code>

stash RDI on stack
 call snek_print²
 RESTORE RDI from stack.

CSE 231: Functions

Ranjit Jhala April 23, 2026

User-defined Functions

Program

- A list of *function definitions*
- A *main expression*

```
<prog> := <func>* <expr>
```

Function Definitions

- A name,
- A parameter (for now),
- A body expression.

```
<func> := (fun (<name> <ident>) <expr>)
```

Expressions

- ... as before
- plus *function calls*

```
<expr> := ... | (<name> <expr>)
```

QUIZ: Abstract Syntax

```
struct Prog {
```

```
}
```

```
struct Defn {
```

```
  name: Str
  param: Str
  body: Expr
```

```
}
```

```
enum Expr {
```

```
}
```

QUIZ: Semantics

Fill in the result of evaluating the following programs.

Program

Result

```
(fun (incr n)
  (add1 n)
)
"main"
(incr 100)
```

101

```
(fun (fac n)
  (if (= n 0)
    1
    (* n (fac (sub1 n)))
  )
)
"main"
(fac 5)
```

$5 \times 4 \times 3 \times 2 \times 1 \times 1$

120

Evaluation $\text{type FunEnv} = \text{Map}(\text{String}, \text{Defn})$

```
fn eval(e: &Expr, env: &mut Env, fenv: &FunEnv)
-> Result<Val, Err>
{
  match e { //
    Call(f, arg) => {
      let f-def = fenv.get(f);
      let val = eval(arg, env, fenv);
      let new-env = Env::new().insert(f-param, val);
      eval(f-body, new-env)
    }
  }
}
```

(let (x₁ ...)
 (let (x₂ ...)
 (... (foo 94) ...)))

Find Defn of 'f'
 Eval arg to get val

static scope
 v
 dynamic scope

PROG

```
(fun f1 ... )
(fun f2 ... )
...
(fun fk ... )
```

expr

ASM

our code here: $\langle \text{expr} \rangle$ $\xrightarrow{\text{compile-expr}}$ $\langle \text{fun } f_1 \dots \rangle$ $\xrightarrow{\text{compile-fn}}$ $\langle \text{fun } f_2 \dots \rangle$ $\dots$ $\langle \text{fun } f_k \dots \rangle$

start-f₁:
 start-f₂:

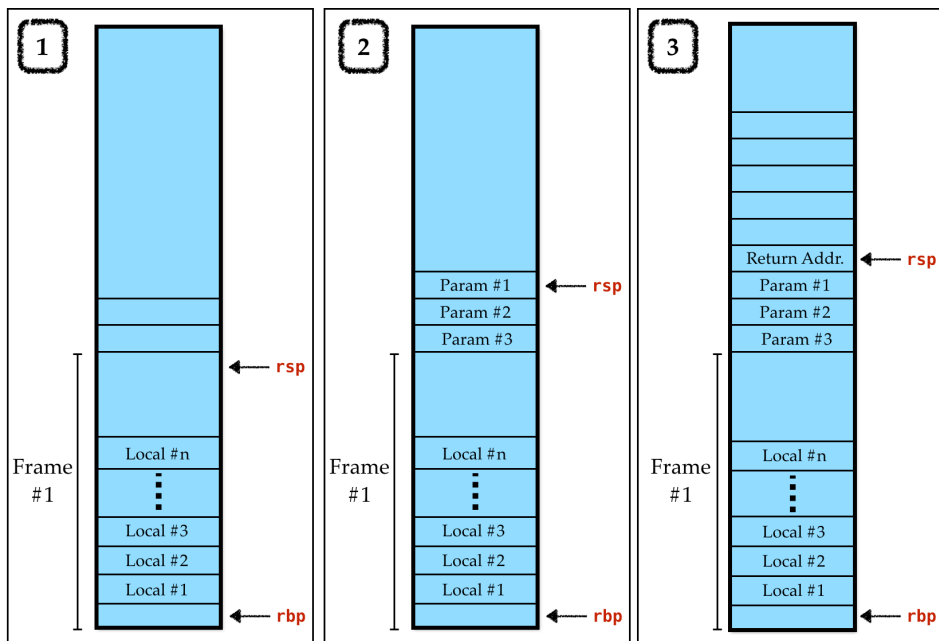
let (x —)

let (x ← 10)
 (let (f (fun a (+ a x)))

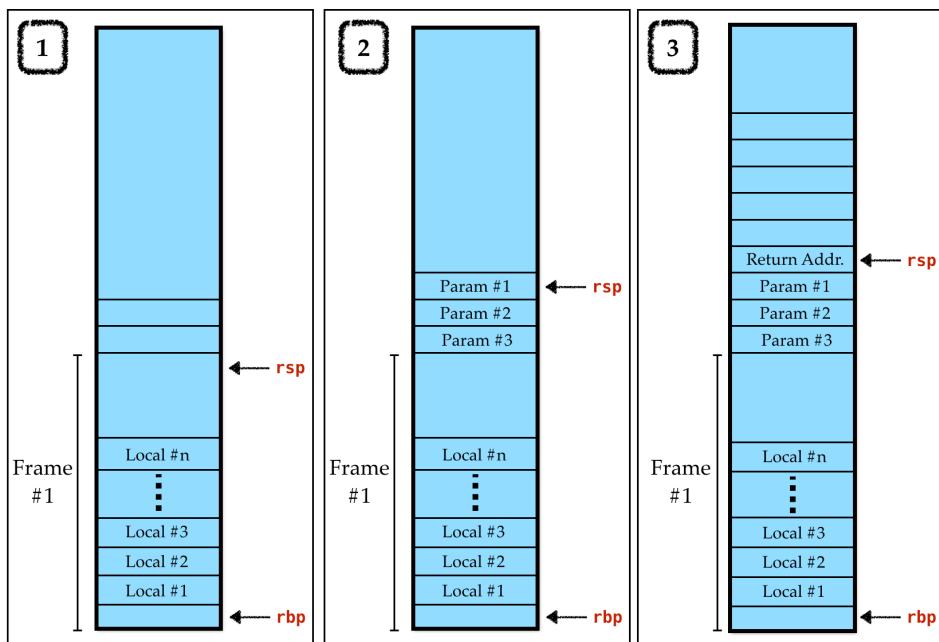
(f 100)

Callers, Callees, and Frames

Caller



Callee



QUIZ: Assembly: Caller

Program

```
(incr 100)
```

Assembly

```
mov rax, 200
```

```
(f e)
```

```
; << e >>
```

QUIZ: Assembly: Callee

Program

```
(fun (incr n)
  (add1 n)
)
```

Assembly

```
; setup frame
```

```
; body
```

```
; teardown frame
```

Compiling Calls

```
fn compile_expr(e: &Expr) -> String {
}
```

Compiling Definitions

```
fn compile_defn(defn: &Defn) -> String {
}
```

Frame Setup (*frame_setup*)

```
fn frame_setup(e: &Expr) -> String {  
  
  
  
  
  
  
}
```

Frame Teardown (*frame_teardown*)

```
fn frame_takedown(e: &Expr) -> String {  
  
}
```

Frame Allocation

How much *space* should we allocate for a frame?

How much should we float rsp up?

```
fn max_vars(e: &Expr) -> usize {
```